Aran Learn 13: Post-Quantum Cryptography

For: Inbathamizhan and Aasif in full; **everyone reads sections 1 to 3** (why this matters for a bank). **Time:** about 5 hours with practice. **Before this:** Learn 3 (Cryptography Basics).

A large enough quantum computer would break the public-key cryptography that banks use today. This doc explains, from zero, what breaks and what does not, what the new standards are, and how Aran was upgraded. Every number here was checked by running the code in `research/`.

1. What a quantum computer changes

A normal computer stores bits, each 0 or 1. A quantum computer stores **qubits**, which can be in a mix of 0 and 1 at once (superposition) and can be linked together (entanglement). For most jobs this gives no speed-up at all. For a few special mathematical problems it gives a huge one, and two of those problems sit under today's internet security.

You do not need the physics. You need one table.

Algorithm	What it does	What it breaks	How badly
Shor (1994)	Factors large numbers and solves discrete logarithms fast	RSA, Diffie-Hellman, elliptic curves: ECDSA, ECDH, **Ed25519, X25519**	Completely. The private key falls out of the public key
Grover (1996)	Searches N possibilities in about the square root of N steps	Symmetric keys and hashes (AES, SHA-256)	Halves the bit strength: a 128-bit key acts like 64 bits, a 256-bit key like 128 bits

So the rule of thumb is short:

- **Public-key crypto (RSA, elliptic curves): replace it.**
- **Symmetric crypto (AES) and hashes (SHA-256): keep it, but use 256-bit keys.**

When? Nobody knows. A machine that can run Shor at full size is called a CRQC (cryptographically relevant quantum computer). The estimate keeps shrinking: in 2019 breaking RSA-2048 was thought to need about 20 million noisy qubits; in May 2025 Craig Gidney showed it could be done with fewer than a million noisy qubits in under a week, assuming a 0.1% error rate. Today's largest machines are far smaller than that, but the gap is narrowing faster than people expected.

2. Why it matters today, before the machine exists

Two attacks work right now, even though the quantum computer comes later.

Harvest now, decrypt later. An attacker records encrypted bank traffic today and stores it. Years later, with a quantum computer, they break the key exchange and read everything. If the

data is still secret by then, the damage is real. An Aadhaar number does not change for life, so an Aadhaar number recorded in 2026 is still worth stealing in 2040.

Forge later. A signature made today with Ed25519 can be forged once Ed25519 is broken. Anything that must stay trustworthy for years (a signed erasure receipt, a contract, a software update) is exposed.

RBI Deputy Governor Shirish Chandra Murmu named harvest-now-decrypt-later directly in a speech at the Global FinTech Fest on 11 September 2026 and urged Indian payment operators to start moving to quantum-safe crypto now.

3. Mosca’s inequality: is my data at risk?

Michele Mosca gave a simple test. Let:

- **x** = how many years the data must stay secret (its shelf life)
- **y** = how many years it will take to migrate your systems
- **z** = how many years until a quantum computer exists

If **x + y > z**, you are already late: some data protected with today’s crypto will still matter after the machine arrives.

Worked example (Aadhaar). Shelf life x = 70 years (it lasts a lifetime). India’s quantum-safe task force (Department of Science and Technology, February 2026) targets full adoption for banking and other standard-track sectors by 2033, so y = 7 years from 2026. Suppose a CRQC arrives in 2035, so z = 9. Then x + y - z = 70 + 7 - 9 = **68 years of exposure**. Even with a late machine (2040, z = 14) it is 63 years. Every Aadhaar number crossing a classical TLS link today is in scope.

Counter-example (OTP). Shelf life is a few minutes. A quantum attack that takes a week is useless against it. OTPs are not a quantum priority.

`research/experiment_pqc.py` (E19) computes this table for every data type Aran protects.

4. The new standards

NIST ran an open competition from 2016 and published the first post-quantum standards on 13 August 2024. India’s task force says it will adopt the same algorithms.

Standard	Name	Job	Built on	Sizes (level 3)
FIPS 203	**ML-KEM** (was Kyber)	Agree on a shared key	Lattices (module learning with errors)	public key 1,184 bytes, ciphertext 1,088 bytes
FIPS 204	**ML-DSA** (was Dilithium)	Digital signatures	Lattices	public key 1,952 bytes, signature 3,309 bytes
FIPS 205	**SLH-DSA** (was SPHINCS+)	Digital signatures, backup choice	Only hash functions	small keys, signatures of 7 to 50 KB
(FIPS 206, in progress)	FN-DSA (was Falcon)	Smaller signatures	Lattices	
(selected March 2025)	HQC	Backup key agreement	Error-correcting codes	

Compare with what they replace: an X25519 public key is 32 bytes and an Ed25519 signature 64 bytes. **Post-quantum crypto is bigger, not slower.** In E18, ML-KEM-768 encapsulation took 28 µs, faster than X25519 (75 µs); the cost is size.

4.1 The idea behind lattices, with a toy you can run

Learning With Errors (LWE) is the hard problem under ML-KEM and ML-DSA. Take a secret list of numbers **s**. Publish many random rows **A** and, for each row, the value A·s **plus a tiny random error e**, all modulo q. Without the errors, ordinary school algebra (Gaussian elimination) recovers s instantly. With the errors, nobody knows a fast way, on normal or quantum computers.

Here is a toy that encrypts single bits (q = 97, 4 secret numbers, 8 rows). Real ML-KEM uses 768-dimensional vectors of polynomials, but the shape is the same.

```
        import random
        rng = random.Random(7)
        q, n, m = 97, 4, 8
        s = [rng.randrange(q) for _ in range(n)]                      # secret key
        A = [[rng.randrange(q) for _ in range(n)] for _ in range(m)]   # public random matrix
        e = [rng.choice([-1, 0, 1]) for _ in range(m)]                # small noise
        b = [(sum(A[i][j] * s[j] for j in range(n)) + e[i]) % q for i in range(m)]  # public
key: (A, b)

        def encrypt(bit):
            r = [rng.randint(0, 1) for _ in range(m)]                 # pick a random subset
of rows
            u = [sum(r[i] * A[i][j] for i in range(m)) % q for j in range(n)]
            v = (sum(r[i] * b[i] for i in range(m)) + bit * (q // 2)) % q
            return u, v

        def decrypt(u, v):
            d = (v - sum(u[j] * s[j] for j in range(n))) % q          # = noise + bit*48
            return 0 if min(d, q - d) < q // 4 else 1, d
```

Output:

```
secret s = [41, 19, 50, 83]
noise e  = [1, -1, 0, 0, -1, 1, -1, 1]
bit 0: v - u.s = 0  -> decrypts to 0
bit 1: v - u.s = 47 -> decrypts to 1
bit 1: v - u.s = 48 -> decrypts to 1
bit 0: v - u.s = 3  -> decrypts to 0
```

The owner of s removes the big part and is left with “small noise” (near 0) or “small noise + 48” (near q/2). Everyone else sees numbers that look random.

5. Key encapsulation (KEM) instead of key exchange

Diffie-Hellman and X25519 are a **key exchange**: both sides mix their secrets and arrive at the same key. ML-KEM is a **KEM**: the receiver publishes a public key; the sender runs `encapsulate`, which returns a fresh shared secret and a ciphertext; the receiver runs `decapsulate` on the ciphertext and gets the same secret. That secret then keys AES-256-GCM. This pattern (KEM for the key, AES for the data) is called KEM-DEM.

```
        from cryptography.hazmat.primitives.asymmetric import mlkem, mldsa

        receiver = mlkem.MLKEM768PrivateKey.generate()
        shared_sender, ciphertext = receiver.public_key().encapsulate()
        shared_receiver = receiver.decapsulate(ciphertext)
        print("same secret:", shared_sender == shared_receiver, "| ciphertext bytes:",
len(ciphertext))

        signer = mldsa.MLDSA65PrivateKey.generate()
        sig = signer.sign(b"erase C00042 Marketing", b"aran-v2")      # second argument: a
context label
        signer.public_key().verify(sig, b"erase C00042 Marketing", b"aran-v2")
        print("signature bytes:", len(sig))
```

```
same secret: True | ciphertext bytes: 1088
signature bytes: 3309
```

This runs with the `cryptography` package Aran already uses (version 50 ships ML-KEM and ML-DSA), so no new library is needed.

6. Hybrid: belt and braces

The new algorithms are young. Lattice schemes have been studied for about 25 years, elliptic curves for about 40. A clever classical attack on ML-KEM is unlikely but not impossible, and implementations can have bugs. So the industry standard during the transition is **hybrid**: run a classical and a post-quantum algorithm together, and stay safe as long as **either one** holds.

Hybrid key agreement. Do X25519 and ML-KEM-768, then hash both shared secrets together (with the ciphertexts and public key, so nothing can be swapped):

```
key = SHA3-256(label || ss_mlkem || ss_x25519 || ct_x25519 || pk_x25519)
```

This is the X-Wing construction. A quantum attacker who breaks X25519 still lacks `ss_mlkem`; a classical attacker who somehow breaks ML-KEM still lacks `ss_x25519`. Browsers and servers already do this in TLS under the name **X25519MLKEM768**: Chrome and Cloudflare turned it on by default in late 2024, Firefox followed, and OpenSSL 3.5 (April 2025) made it the default too.

Hybrid signatures. Sign the same bytes with Ed25519 and ML-DSA-65, and accept only if **both** verify. A forger must break both.

The downgrade trap. If a verifier also accepts old Ed25519-only signatures, an attacker who breaks Ed25519 just deletes the ML-DSA half and presents the result as “old”. Two defences, both in Aran: (1) the algorithm name is inside the signed bytes, so removing it breaks the signature; (2) a `require_pq` switch that refuses Ed25519-only signatures once the bank decides Ed25519 is no longer enough.

7. Crypto-agility and the CBOM

You cannot migrate what you cannot find. A **CBOM** (cryptographic bill of materials) lists every algorithm a product uses, where, and how strong it is. India's task force asks technology vendors to supply CBOMs from 2027 to 2028. The CycloneDX 1.6 format has a standard place for this.

Crypto-agility means writing the algorithm name next to every signature, ciphertext and key (`"alg": "Ed25519+ML-DSA-65"`), so the next change of algorithm is a configuration change, not a rewrite. Aran's receipts and key backups now carry an `alg` field.

8. What changed in Aran

Part	Before	Quantum status before	After
FF1 tokens (M1, M3)	AES-128 key in the live console	Grover: 64 bits	AES-256 key: 128 bits after Grover. Checked against NIST FF1 samples 7 and 8
Vault encryption (M5)	AES-256-GCM	Safe	Unchanged
HMAC registries, Merkle log	SHA-256	Safe	Unchanged
Erasure receipts (M5)	Ed25519	**Broken by Shor**	Ed25519 + ML-DSA-65, both required; old receipts still verify unless `require_pq=True`
Purpose keys leaving the vault (backup, DR)	Not specified; usually X25519 or RSA	**Broken by Shor**	X25519 + ML-KEM-768 hybrid seal (`pqc/hybrid.py`)
Inventory	None		`python -m pqc.cbom` writes a CycloneDX CBOM

The most important finding is in the research doc: **crypto-shredding can be undone retroactively** if a destroyed key was ever sent anywhere under X25519. In E17 an attacker who recorded one night's key backup recovered 100% of the 1,200 erased values after a simulated break of X25519, and 0% when the backup was sealed with the hybrid.

9. Practice

1. Run the toy LWE code with the noise removed (`e = [0]*m`). Show that s can then be recovered from (A, b) with Gaussian elimination modulo 97. Why does adding noise stop this?
2. Make the noise bigger: `rng.choice(range(-15, 16))`. Encrypt 1,000 random bits. What fraction decrypt wrongly, and why? (This is why real schemes choose q and the noise size carefully.)
3. Fill in Mosca's inequality for these, with y = 7: a home-loan file kept 30 years after the loan ends; a CVV; a signed software update for an ATM that stays in service 12 years. Which need hybrid crypto first?
4. Run `cd research && python -m pqc.cbom ..` and explain every line marked "depends on key size". Find the key-generation line for each one and check it is 256 bits.

5. In `research/test_pqc.py`, read `test_legacy_receipts_still_verify_but_downgrade_is_blocked`. Write one more test: change the `alg` field of a hybrid receipt to `"Ed25519"`. What should happen, and does it?
6. Configure nginx for hybrid TLS (needs OpenSSL 3.5 or newer): add `ssl_protocols TLSv1.3;` and `ssl_ecdh_curve X25519MLKEM768:X25519;`. Connect with a current Chrome and open the Security panel of DevTools: the key exchange should read X25519MLKEM768.

Hints. (1) With e = 0, b = A·s exactly: four independent rows give four equations in four unknowns. (2) Decryption fails once the summed noise passes q/4 = 24. (3) Loan file 30 + 7 > z for any z under 37: yes. CVV: shelf life is the card's life, a few years, lower priority. ATM update: signatures must stay unforgeable for 12 years, so 12 + 7 = 19 > z for most estimates: yes, sign with a hybrid or SLH-DSA. (5) The `alg` value is part of the signed bytes; changing it breaks both signatures, and `"Ed25519"` is not a known hybrid name, so it fails.

Sources

- P. W. Shor, "Algorithms for Quantum Computation: Discrete Logarithms and Factoring", FOCS 1994. L. K. Grover, "A Fast Quantum Mechanical Algorithm for Database Search", STOC 1996.
- C. Gidney, "How to factor 2048 bit RSA integers with less than a million noisy qubits", arXiv:2505.15917, May 2025.
- NIST FIPS 203 (ML-KEM), FIPS 204 (ML-DSA), FIPS 205 (SLH-DSA), 13 August 2024. NIST IR 8547 (initial public draft, November 2024): quantum-vulnerable public-key algorithms deprecated after 2030, disallowed after 2035.
- O. Regev, "On Lattices, Learning with Errors, Random Linear Codes, and Cryptography", STOC 2005.
- M. Mosca, "Cybersecurity in an Era with Quantum Computers: Will We Be Ready?", IEEE Security & Privacy, 2018.
- M. Barbosa et al., "X-Wing: The Hybrid KEM You've Been Looking For", IACR Communications in Cryptology, 2024. IETF draft "Hybrid key exchange in TLS 1.3" (X25519MLKEM768).
- Department of Science and Technology, Government of India, "Implementation of Quantum Safe Ecosystem in India: Report of the Task Force", February 2026 (summary: https://postquantum.com/security-pqc/indias-quantum-safe-roadmap/).
- RBI Deputy Governor S. C. Murmu, remarks at Global FinTech Fest, 11 September 2026 (as reported at https://www.qnulabs.com/blog/rbi-quantum-proof-payment-systems-2026).
- CycloneDX 1.6 specification, cryptographic bill of materials (CBOM).
- Free reading: Cloudflare blog posts on post-quantum TLS; the Open Quantum Safe project (openquantumsafe.org).